CONSTITUTION_INTEGRATION

Constitution Submodule Integration & Amendment Specification

Field	Value
Revision	2
Created	2026-07-24
Last modified	2026-09-22
Status	active
Status summary	SpecKit-Superpowers Bridge constitutional integration plan — new anchor proposal §11.4.NNB (UNLANDED), governance-file amendment catalogue, consumer-onboarding recipe, and per-§-letter binding table. **2026-09-22 (HXC-159 Phase B3): the §2 anchor proposal's implicit dependency on the never-built nano-task-engine/"SuperBridge MCP" infrastructure is flagged and corrected — see the amendment note at §2.**
Issues	none
Issues summary	—
Fixed	initial document + HXC-159 Phase B3 architecture-correction pass (§2 amendment note)
Fixed summary	This document codifies how the SpecKit-Superpowers Bridge integrates with the Helix Constitution submodule (git@github.com:HelixDevelopment/helix_constitution.git).
Continuation	

Field	Value
	Awaiting constitution anchor allocation; gate-code = separate PWU per §11.4.227.

> **ARCHITECTURE AMENDMENT — 2026-09-22 (HXC-159 Phase B3).** This project's REAL Bridge architecture has been corrected to 6 layers with no MCP execution tier — "SuperBridge MCP" was independently re-derived twice (GitHub `total_count: 0`; npm exact-name lookup `HTTP 404`) not to exist as software. See `../README.md#architecture-amendment-2026-09-22` for the full finding and the worked proof (Spec Kit + Superpowers + Claude Code's own skill/subagent machinery). The §2 anchor proposal below is corrected in-place (see its own amendment note); the remainder of this document is unaffected design content.

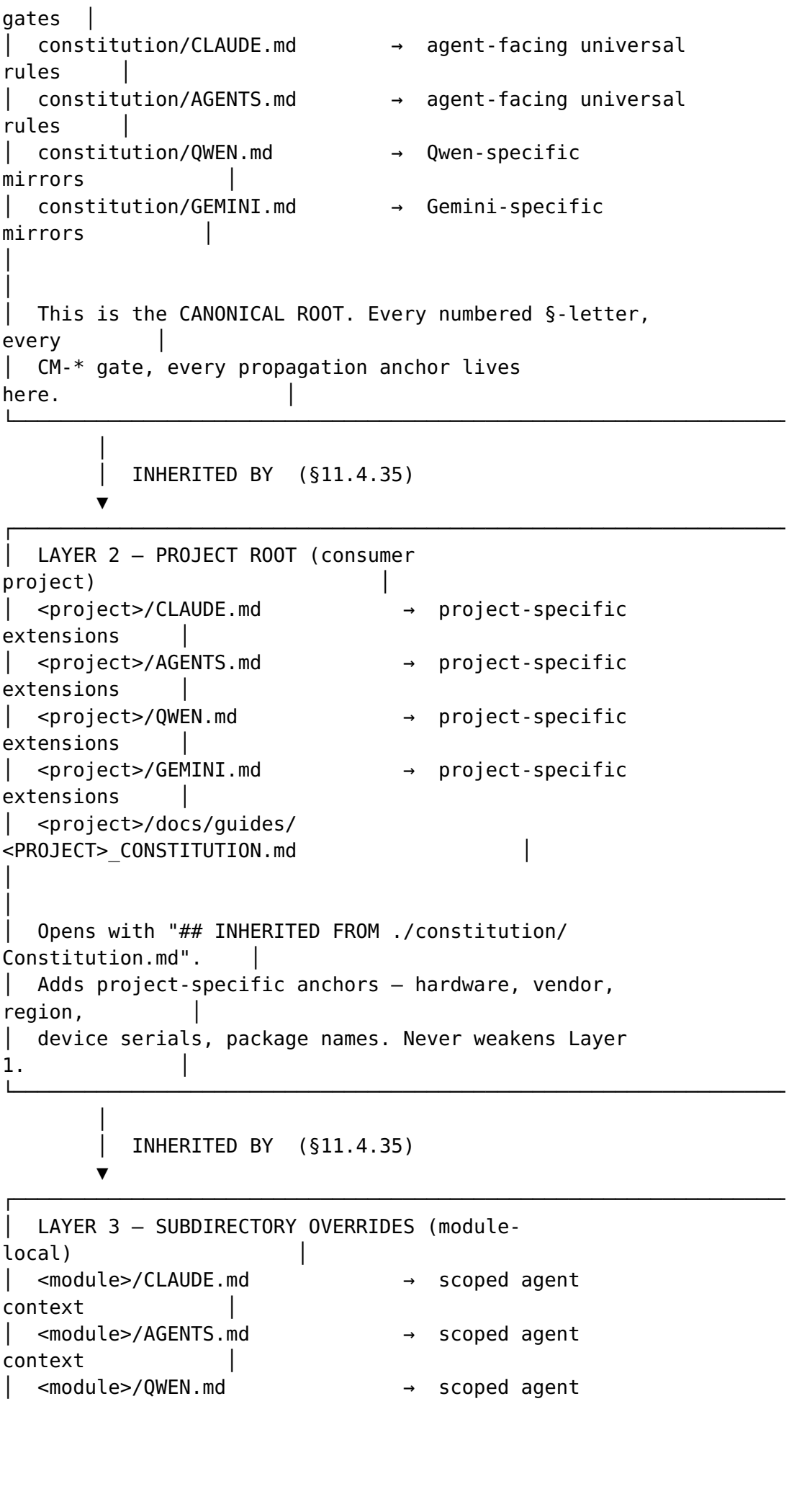
Table of contents

- §1. Three-Layer Governance Model
- §2. New Constitution Anchor Proposal — SpecKit-Superpowers Lifecycle
- §3. Consumer Onboarding — Step-by-Step
- §4. Constitution Consumption Workflow
- §5. Extension Auto-Discovery
- §6. Per-Project Configuration — DATA vs ENGINE
- §7. Docs Chain Integration
- §8. Auto-Propagation Hook
- §9. Constitutional Mandates Binding the Bridge
- §10. Governance File Amendments
- §11. Catalogue Registration
- §12. Validation & Verification
- §13. Implementation Phases

§1. Three-Layer Governance Model

The Helix Constitution operates a three-layer rule hierarchy (§11.4.17, §11.4.35). Every project incorporating the constitution as a git submodule inherits rules from all three layers, with lower layers tightening (never weakening) higher ones.

```
┌──────────────────────────────────────────────────────────────────────┐
│  LAYER 1 — BASE (constitution submodule)                             │
│  constitution/Constitution.md  →  universal rules, anchors,
```


gates
constitution/CLAUDE.md → agent-facing universal rules
constitution/AGENTS.md → agent-facing universal rules
constitution/QWEN.md → Qwen-specific mirrors
constitution/GEMINI.md → Gemini-specific mirrors
This is the CANONICAL ROOT. Every numbered §-letter, every CM-* gate, every propagation anchor lives here.
INHERITED BY (§11.4.35)
LAYER 2 — PROJECT ROOT (consumer project)
<project>/CLAUDE.md → project-specific extensions
<project>/AGENTS.md → project-specific extensions
<project>/QWEN.md → project-specific extensions
<project>/GEMINI.md → project-specific extensions
<project>/docs/guides/<PROJECT>_CONSTITUTION.md
Opens with "## INHERITED FROM ./constitution/Constitution.md".
Adds project-specific anchors — hardware, vendor, region, device serials, package names. Never weakens Layer 1.
INHERITED BY (§11.4.35)
LAYER 3 — SUBDIRECTORY OVERRIDES (module-local)
<module>/CLAUDE.md → scoped agent context
<module>/AGENTS.md → scoped agent context
<module>/QWEN.md → scoped agent

```
context            |
|  <module>/GEMINI.md            →  scoped agent
context            |
|
|
|  Tightens (never weakens) rules for one directory/sub-
module.      |
|  Example: prebuilt_apps/CLAUDE.md adds an app-specific
constraint. |
└──────────────────────────────────────────────────────────────────────────┘
```

How a CLI agent resolves rules

When an agent's working directory is `<project>/sub/module/`, it walks **upward** reading each governance file it finds, stacking rules from most-specific (deepest) to most-general (repo root), then finally the constitution submodule:

```
</mnt/track1/atmosphere-t1/sub/module/CLAUDE.md>     ← Layer 3
(deepest)
</mnt/track1/atmosphere-t1/CLAUDE.md>                 ← Layer 2
(project)
</mnt/track1/atmosphere-t1/constitution/CLAUDE.md>    ← Layer 1
(universal)
```

The SpecKit-Superpowers Bridge anchor (§11.4.NNB) lives at **Layer 1** — every project inherits it automatically. Project-specific configuration (DB paths, hardware specs, LLM preferences, design tokens) lives at **Layer 2** as data the engine consumes.

---

§2. New Constitution Anchor Proposal — SpecKit-Superpowers Lifecycle

> **ARCHITECTURE AMENDMENT — 2026-09-22 (HXC-159 Phase B3).** This anchor proposal is **UNLANDED** — it has never been adopted into `Constitution.md`; the anchor number itself is explicitly a placeholder ("the actual number is assigned by the constitution maintainer on landing"). It calls the whole Bridge a mandatory "core technology stack" including the nano-task decomposition & execution engine described elsewhere in this corpus (`NANO_TASK_ENGINE.md`, `EXTENSION_DEVELOPMENT.md` Part C) — and that engine's own design assumed a "SuperBridge MCP" execution tier which has been independently re-derived TWICE (GitHub `total_count: 0`; npm exact-name lookup `HTTP 404`) **not**

> **to exist as software**. **Verified 2026-09-22:** the pipeline text below (stage 5, IMPLEMENT) does NOT itself invoke an MCP server — it already specifies subagent-driven execution in an isolated git worktree (§11.4.70/§11.4.58 L4), which is consistent with the corrected substitution. The correction needed here is narrower than a rewrite: **if this proposal is ever pursued for landing**, its "core technology stack" framing MUST be corrected to drop any implicit dependency on the never-built nano-task-engine/MCP infrastructure, and its execution substrate MUST be stated explicitly as **Spec Kit** + **Superpowers** + **Claude Code's own skill/subagent machinery** (proven, not merely asserted, by the T-B3.02 worked run — see `../README.md#architecture-amendment-2026-09-22`) rather than any MCP-server-mediated tool-contract layer. Until landed, nothing below binds any project; it is retained design intent per §11.4.35/§11.4.124.

Proposed anchor number

`§11.4.NNB` — placeholder; the actual number is assigned by the constitution maintainer on landing. The next free number in the §11.4 sequence is requested.

Anchor text

§11.4.NNB — SpecKit-Superpowers Bridge lifecycle: every governed project MUST ship the bridge as a core technology stack, auto-discovered from the constitution submodule, driving specify→plan→tasks→implement→review through constitutionally-bound nano-task layers with decoupled subagent execution and full TDD coverage (User mandate, 2026-07-24).

The SpecKit-Superpowers Bridge is a **constitutionally-bound core technology stack** — derived to every consuming project from the constitution submodule — that orchestrates the full feature-development lifecycle from specification through delivery. It is NOT an optional plugin, NOT a per-project re- implementation, and NOT a carve-out from any existing constitutional mandate.

(A) Bridge lifecycle. Every governed feature passes through the 6-stage bridge pipeline:

1. **SPECIFY** — a natural-language feature description becomes a formal `spec.md` (from the `speckit-specify` skill). The spec carries §11.4.44 revision header + §11.4.91 ≥40-char meaningful descriptions for every sub-section. It is exported to `.html+.pdf+.docx` per §11.4.65.

2. **CLARIFY** — underspecified areas are identified and the operator is asked ≤5 highly targeted questions per §11.4.66 interactive-clarification mechanism; answers are encoded back into the spec (the `speckit-clarify` skill).

3. **PLAN** — a `plan.md` is generated (the `speckit-plan` skill), containing architecture, data-flow, contract definitions, and a constitution-compliance cross-check enumerating every applicable §-letter with per-section coverage. The plan cites §11.4.8 research references AND §11.4.150 multi-angle deep- research passes for every non-trivial design decision. The plan is run through the `speckit-analyze` skill for non-destructive cross-artifact consistency analysis per §11.4.186 consistency enforcement.

4. **TASKS** — a `tasks.md` is generated (the `speckit-tasks` skill), decomposed into the **finest granularity decoupled nano-tasks** the bridge can produce. Every task is: (i) independently implementable by a single subagent (§11.4.20 / §11.4.70) with zero coupling to any other in-flight task; (ii) preceded by its own RED test (§11.4.43 TDD-fix-discipline, §11.4.115 RED-baseline-on-the-broken-artifact, §11.4.224 test-first for ALL work); (iii) scoped to ≤30 minutes wall-clock (§11.4.82(G) subagent scope discipline); (iv) carrying explicit dependency edges to other tasks (a DAG) so assembly tasks bind results without re-executing; (v) registered as a tracked workable item (§11.4.54 ATM-NNN id, §11.4.15 Status=Queued, §11.4.16 Type=Task) in the project's SQLite SSoT (§11.4.93/§11.4.95).

5. **IMPLEMENT** — tasks execute through the `speckit-implement` skill via subagent-driven development (§11.4.70), each subagent running in an isolated `git worktree` (§11.4.58 L4 / §11.4.84), registered in the agent-lifecycle tracker (§11.4.147 crashed-agent respawn), with every closure landing four-layer test coverage (§11.4.4(b): pre-build gate + post-build gate + on-device runtime test + paired §1.1 mutation).

6. **REVIEW** — the implemented batch passes through the full review gauntlet: independent code-review on Fable-xhigh (§11.4.125 / §11.4.142 / §11.4.209), iterated to ZERO findings + ZERO warnings (§11.4.134), with §11.4.194 exhaustive all-scenario analysis and §11.4.145 eight-angle impact research.

(B) Nano-task decoupling. Tasks are decomposed to the **smallest self-contained work unit** the language and feature permit — so small that even a weaker local LLM (operating within host hardware constraints) can execute each flawlessly. Assembly tasks compose decoupled results without re-implementing them. The nano-task graph is layered: leaf tasks produce primitive artifacts, mid-layer tasks compose them, and top-layer tasks wire the composed results into the application. Every task carries explicit input/output contracts. Task-to-task coupling is structural (declared file-scope manifest §11.4.58) — never assumed/improvised (§11.4.6).

(C) TDD at every layer. The TDD cycle binds every task:

TDD Phase	Constitutional Binding
RED — test FIRST, observed to FAIL	§11.4.43 step 1 + §11.4.115 `RED_MODE=1` + §11.4.224(A)
GREEN — implementation makes it pass	§11.4.43 step 3 + §11.4.115 `RED_MODE=0` polarity switch
VERIFY — re-run, deterministic consistency	§11.4.50 N=3 iters, same frozen artifact fingerprint
DOCUMENT — Issues→Fixed migration	§11.4.43 step 5 + §11.4.33 type-aware closure

A RED authored AFTER implementation is a §11.4 PASS-bluff — the bridge MUST produce RED-first evidence (§11.4.224(A) + §11.4.115(F) machine-written verdict pairs).

(D) Constitutional anti-bluff at every stage. Every bridge stage MUST produce captured physical evidence per §11.4.5 / §11.4.69 / §11.4.107 — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden at every layer (§11.4 / §11.4.1). The `speckit-analyze` skill's cross-artifact consistency analysis is self-validated golden-good/golden-bad per §11.4.107(10). A bridge stage that produces a green signal without captured evidence is a §11.4 bluff at the lifecycle-orchestration layer.

(E) Auto-discovery. The bridge is inherited from the constitution submodule as a core technology stack (§11.4.74 extend-don't-reimplement). Skills are auto-discovered via the §11.4.164 `post_update_hook.sh` propagation seam and registered in the consuming project's skill index. No per-project installation step beyond `git submodule update --remote constitution` is required.

(F) Per-project configuration. The bridge ENGINE carries ZERO project literals (§11.4.28 decoupling). Consuming projects supply their configuration as DATA: LLM provider preferences, hardware capacity (cores, RAM, VRAM), project-specific ID prefixes, tracker bindings, and design-token paths (§11.4.35).

(G) Composition with existing anchors. The bridge COMPOSES — never weakens — every anchor it touches. Its role is to mechanize what the existing anchors mandate as prose: it takes "every fix SHALL have a RED test" and makes it the mechanical output of the plan→tasks→implement pipeline rather than a discipline the agent must remember.

Classification: **universal** (§11.4.17) — the SpecKit-Superpowers lifecycle is reusable by ANY governed project, carrying zero project-specific hardware, vendor, or region assumptions. Consuming projects supply their per-project DATA as described in §6.

Composition mapping

The anchor composes with (does NOT weaken, does NOT re-define):

§-letter	What the bridge adds
§11.4.17	Universal-vs-project — the bridge is Layer 1 (universal), per-project config is Layer 2 data
§11.4.20	Subagent-driven-by-default — every IMPLEMENT task dispatches as a subagent
§11.4.28	Decoupling — bridge ENGINE is project-agnostic; consumer supplies DATA
§11.4.35	Canonical-root inheritance — bridge rules travel through the 3-layer hierarchy
§11.4.43	TDD-fix-discipline — bridge mechanizes the RED→GREEN→VERIFY→DOCUMENT cycle
§11.4.50	Deterministic consistency — bridge tasks run N=3 iterations on identical artifact
§11.4.58	

§-letter	What the bridge adds
	Parallel-development PWU — every task is a PWU with file-scope manifest
§11.4.66	Interactive clarification — CLARIFY stage encodes answers back into spec
§11.4.70	Subagent-driven execution — IMPLEMENT stage defaults to subagents
§11.4.74	Catalogue-first — bridge skills registered in submodules-catalogue.md
§11.4.84	Working-tree quiescence — tasks use isolated git worktrees
§11.4.85	Stress+chaos — every fix ships with resilience tests
§11.4.107	Anti-bluff AV — bridge stages produce captured evidence, not metadata-only
§11.4.115	RED-baseline + polarity — every task carries `RED_MODE=1→0`
§11.4.125	Code-review gate — REVIEW stage dispatches independent reviewers
§11.4.134	Iterate-until-GO — REVIEW iterates to ZERO findings + ZERO warnings
§11.4.142	Universal review — every task diff crosses independent review
§11.4.145	Impact research — PLAN stage includes 8-angle impact analysis
§11.4.146	Reproduce-first-then-extend — each task follows the 3-step workflow
§11.4.147	Crashed-agent respawn — every IMPLEMENT subagent tracked in lifecycle registry
§11.4.150	Deep multi-angle research — PLAN cites research passes per §11.4.8
§11.4.164	Auto-propagation — bridge skills installed via post_update_hook.sh
§11.4.186	Cross-document consistency — speckit-analyze enforces spec↔plan↔tasks consistency
§11.4.194	Exhaustive code-review — REVIEW covers all scenarios + captured evidence cross-check
§11.4.197	Research completion — every SPECIFY→PLAN artifact maps to a tracked workable item
§11.4.209	

§-letter	What the bridge adds
	Fable-xhigh review — REVIEW dispatches on Fable model at xhigh effort
§11.4.224	TDD for ALL work — every task (including new features) ships RED-first
§11.4.227	Governance self-custody — the bridge anchor's named gates are registered in the ledger

Propagation gate name

`CM-COVENANT-114-NNB-PROPAGATION` — asserts the literal `11.4.NNB` is present (block-start shaped per §11.4.227(B)) in every governance mirror file across the 5-carrier lockstep set (Constitution.md + CLAUDE.md + AGENTS.md + QWEN.md

- GEMINI.md, §11.4.157).

Recommended mechanism gates

Gate	What it enforces
`CM-SPECKIT-BRIDGE-SKILLS-PRESENT`	All 8 bridge skills (speckit-specify, speckit-plan, speckit-tasks, speckit-implement, speckit-analyze, speckit-clarify, speckit-checklist, speckit-converge) exist + parse-clean in the constitution skills directory
`CM-SPECKIT-BRIDGE-AUTO-DISCOVERY`	`post_update_hook.sh` detects bridge-skill changes and registers them in consuming projects
`CM-SPECKIT-BRIDGE-TDD-WIRED`	Every IMPLEMENT task carries a RED-first verdict pair (§11.4.115(F)) + observed precondition-provenance field
`CM-SPECKIT-BRIDGE-NANO-DECOUPLING`	Task graph produces decoupled nano-tasks with explicit contracts; no task exceeds §11.4.82(G) 30-min wall-clock budget
`CM-SPECKIT-BRIDGE-CATALOGUE-REGISTERED`	Bridge registered in `submodules-catalogue.md` under its capability category
`CM-SPECKIT-BRIDGE-`	Per-project config file schema documented + `report_item.sh` validates consumer config every run

Gate	What it enforces
CONSUMER-CONFIG	

All gates ship paired §1.1 meta-test mutations. Gate-code = separate work item per §11.4.227(A).

---

§3. Consumer Onboarding — Step-by-Step

A new project that incorporates the Helix Constitution and wants to use the SpecKit-Superpowers Bridge follows this bootstrap sequence:

Step 1 — Add the constitution submodule

```
# In the project root
git submodule add git@github.com:HelixDevelopment/
        helix_constitution.git constitution
cd constitution && git checkout v1.0.0 && cd ..
```

Step 2 — Wire the inheritance pointer

```
echo "## INHERITED FROM ./constitution/Constitution.md" >>
        CLAUDE.md
echo "" >> CLAUDE.md
```

The consuming project's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`, and `GEMINI.md` each open with this pointer. Agents walking up from the working directory read the project's governance first, then the constitution's. The constitution's files are authoritative for any topic the project does not cover.

Step 3 — Run the post-update hook

```
bash constitution/scripts/post_update_hook.sh
```

This single invocation:

1. Detects every skill, MCP config, git hook, and action directive that changed since the last constitution pull (or on the initial install, treats everything as "new").
2. Symlinks the bridge skills (`constitution/skills/speckit-*`) into the project's skills directory.

3. Registers the bridge slash commands (`/speckit.specify`, `/speckit.plan`, etc.) in every CLI agent's command set.
4. Installs the git hooks that enforce §11.4.75 mechanical enforcement.
5. Validates all installed scripts (`bash -n` + `sh -n` per §11.4.67).

Step 4 — Supply project-specific configuration

Create the consumer-owned config file at the project-declared path (§11.4.35):

```
mkdir -p config/speckit
```

Example `config/speckit/bridge.yaml`:

```
# Per-project bridge configuration (DATA, not engine code — §11.4.28)
project:
  name: "my-project"                          # Lowercase snake_case (§11.4.29)
  prefix: "${HELIX_RELEASE_PREFIX:-my_project}"  # §11.4.151

hardware:
  cores: 64
  ram_gb: 256
  vram_gb: 32
  model: "threadripper"

llm:
  primary_provider: "local"
  local_backend: "llama.cpp"
  fallback_providers: ["anthropic", "deepseek"]
  preferred_effort: "high"

tracker:
  db_path: "docs/workable_items.db"           # §11.4.95
  id_prefix: "PROJ"                           # §11.4.54
  sync_command: "bash scripts/sync_docs.sh"  # §11.4.106 docs_chain

design:
  tokens_path: "design/opendesign/tokens.css"  # §11.4.216
  brand_colors:
    primary: "#B6E376"
    accent: "#4A90D9"
```

Step 5 — Verify the installation

```
bash constitution/scripts/post_update_hook.sh --verify-only
```

This dry-run validates that all bridge skills are symlinked, all slash commands resolve, and the consumer config parses. It exits 0 only when the installation is complete. The consuming project's pre-build gate `CM-CONSTITUTION-PULL-VALIDATED` (§11.4.32) runs this check automatically.

Step 6 — Start using the bridge

From this point, the CLI agent recognizes the SpecKit-Superpowers lifecycle automatically. The operator types:

```
/SPECKIT.SPECIFY a real-time collaborative whiteboard with
WebSocket sync
```

…and the 6-stage pipeline begins: specify → clarify → plan → tasks → implement → review — all driven by the bridge, all constitutionally-bound, all producing captured physical evidence at every stage.

---

§4. Constitution Consumption Workflow

How a CLI agent inherits rules

When an agent starts in a directory inside a governed project, it resolves governance by walking **upward** from the working directory to the filesystem root, reading every `CLAUDE.md` / `AGENTS.md` it encounters:

```
# Agent working directory: /mnt/track1/atmosphere-t1/device/
rockchip/rk3588/

1. /mnt/track1/atmosphere-t1/device/rockchip/rk3588/CLAUDE.md  ←
deepest (if exists)
2. /mnt/track1/atmosphere-t1/device/rockchip/CLAUDE.md          ←
parent (if exists)
3. /mnt/track1/atmosphere-t1/device/CLAUDE.md                   ←
parent (if exists)
4. /mnt/track1/atmosphere-t1/CLAUDE.md                          ←
project root (Layer 2)
5. /mnt/track1/atmosphere-t1/constitution/CLAUDE.md             ←
constitution (Layer 1)
```

If a project-level file opens with:

```
## INHERITED FROM ./constitution/Constitution.md
```

…the agent reads the constitution's file next. If a deeper file tightens a rule (e.g., "in this subdirectory, additionally enforce X"), the tightest rule wins per the authority hierarchy (§11.4.35).

What the bridge adds to this flow

The bridge skills are inherited from **Layer 1** (the constitution submodule). They are load-bearing constitution features — not project-specific add-ons. A new project inherits all bridge capabilities the moment it runs `post_update_hook.sh` after adding the constitution submodule.

The `find_constitution.sh` helper (shipped in the constitution submodule root) provides the resolved path for any governed project:

```
CONST_DIR=$(bash constitution/find_constitution.sh)
echo "Constitution at: $CONST_DIR"
# → /mnt/track1/atmosphere-t1/constitution
```

Agents and bridge scripts read this value instead of guessing a relative path (§11.4.6 no-guessing).

---

§5. Extension Auto-Discovery

The catalogue system (§11.4.74)

The constitution mandates that every reusable capability under the `vasic-digital` + `HelixDevelopment` organizations is catalogued in `constitution/submodules-catalogue.md`. Before scaffolding any new module, agents MUST survey this catalogue. If a capability exists, the agent MUST reuse or extend it — never reimplement.

How the bridge registers

The SpecKit-Superpowers Bridge registers in the catalogue under the **AI / Agent Tooling** category (mirroring existing entries like `LLMOrchestrator`, `SkillRegistry`, `VisionEngine`):

```
- **[`speckit-bridge`](https://github.com/HelixDevelopment/
      speckit-bridge)** —
  SpecKit→Superpowers lifecycle bridge:
      specify→plan→tasks→implement→review
```

```
  pipeline with constitutional TDD, nano-task decoupling, and
        subagent-driven
  execution
```

The catalogue entry is the single referent for "does a SpecKit bridge already exist?" — agents answering that question read this entry, never re-survey GitHub (per §11.4.74 catalogue-first).

Skill auto-registration

The `post_update_hook.sh` (§11.4.164) detects when the constitution's `skills/` directory gains new entries and registers them in consuming projects:

```
constitution/skills/speckit-specify/SKILL.md       →
<project>/.claude/skills/speckit-specify/
constitution/skills/speckit-plan/SKILL.md          →
<project>/.claude/skills/speckit-plan/
constitution/skills/speckit-tasks/SKILL.md         →
<project>/.claude/skills/speckit-tasks/
constitution/skills/speckit-implement/SKILL.md     →
<project>/.claude/skills/speckit-implement/
constitution/skills/speckit-analyze/SKILL.md       →
<project>/.claude/skills/speckit-analyze/
constitution/skills/speckit-clarify/SKILL.md       →
<project>/.claude/skills/speckit-clarify/
constitution/skills/speckit-checklist/SKILL.md     →
<project>/.claude/skills/speckit-checklist/
constitution/skills/speckit-converge/SKILL.md      →
<project>/.claude/skills/speckit-converge/
constitution/skills/speckit-superpowers-bridge/SKILL.md →

<project>/.claude/skills/speckit-superpowers-bridge/
```

Each symlink is validated: source exists, target is a regular file, symlink resolves. A pre-existing directory at the target path is left untouched (never deleted, §11.4.122).

Slash-command generation

The `scripts/install_cli_agent_plugins.sh` helper (invoked by `post_update_hook.sh`) generates per-agent slash commands from the `constitution/plugins/helix/commands/` templates. The bridge extensions add these commands:

Command	Plugin source	What it does
`/speckit.specify`		

Command	Plugin source	What it does
	`plugins/helix/commands/speckit-specify.md`	Start the SPECIFY stage
`/speckit.clarify`	`plugins/helix/commands/speckit-clarify.md`	Run the CLARIFY stage
`/speckit.plan`	`plugins/helix/commands/speckit-plan.md`	Run the PLAN stage
`/speckit.tasks`	`plugins/helix/commands/speckit-tasks.md`	Run the TASKS stage
`/speckit.implement`	`plugins/helix/commands/speckit-implement.md`	Run the IMPLEMENT stage
`/speckit.analyze`	`plugins/helix/commands/speckit-analyze.md`	Cross-artifact consistency analysis
`/speckit.bridge`	`plugins/helix/commands/speckit-bridge.md`	Full pipeline orchestration

Commands are generated for every CLI agent the project supports (Claude Code, Qwen Code, Gemini CLI — per §11.4.157). The `DEFAULT::` namespace prefix ensures unambiguous resolution even when a host has a built-in `/specify` or `/plan` command (§11.4.140 conflict rule).

---

§6. Per-Project Configuration — DATA vs ENGINE

§11.4.28 mandates that shared submodules carry ZERO project-specific context. The SpecKit-Superpowers Bridge follows this discipline strictly:

What the ENGINE provides (constitution submodule, inherited by reference)

Component	Path in constitution	Description
Bridge skills (8)	`skills/speckit-*/SKILL.md`	Skill definitions + workflows
Plugin commands	`plugins/helix/commands/speckit-*.md`	Slash-command templates
Lifecycle orchestrator	`scripts/speckit/bridge_lifecycle.sh`	

Component	Path in constitution	Description
		Pipeline conductor (orchestrates 6 stages)
Task decomposer	`scripts/speckit/nano_task_decomposer.sh`	Produces decoupled task DAG
TDD harness	`scripts/speckit/tdd_harness.sh`	RED-first + verdict-pair generator
Spec validator	`scripts/speckit/spec_validator.sh`	Validates spec→plan→tasks consistency
Configuration schema	`scripts/speckit/schemas/bridge_config_schema.yaml`	JSON Schema for consumer config
Helix-deps manifest	`helix-deps.yaml`	Declares dependencies under §11.4.31

ALL of the above carry ZERO project literals — no hardcoded paths, device serials, package names, LLM provider API keys, or region assumptions.

What the CONSUMER supplies (project-owned DATA)

Configuration	Example	Constitutional reference
DB path	`docs/workable_items.db`	§11.4.95 (tracked workable-items DB)
ID prefix	`ATM` → item `ATM-001`	§11.4.54 (ticket identifier)
Sync commands	`bash scripts/sync_issues_docs.sh`	§11.4.106 (docs_chain)
Tracker bindings	ClickUp list `901818394542` + field map	§11.4.148 D5 (external tracker sync)
Hardware capacity	`cores: 64 / ram_gb: 256 / vram_gb: 32`	§12.6 (memory budget) + §12.12 (thread limit)
LLM provider preferences	`primary: local / fallback: [anthropic, deepseek]`	§11.4.196 (native-alias-first priority)

Configuration	Example	Constitutional reference
Design tokens	`design/opendesign/tokens.css`	§11.4.216 (canonical design-token source)
Project name	`my_project` (snake_case)	§11.4.29 + §11.4.151 (release prefix)
Default assignee	`@operator_handle` from `.env` var	§11.4.104 (participant attribution)
Default timezone	`Europe/Moscow`	§11.4.208 (request-history timezone)

The engine reads this configuration at runtime. It FAILS CLOSED with an actionable error when configuration is absent — it never guesses (§11.4.6) and never silently defaults to a hardcoded value.

Example consumer config file

```yaml
# <project>/config/speckit/bridge.yaml — consumer-owned DATA
# Read by constitution/scripts/speckit/bridge_lifecycle.sh
# This file is TRACKED in the project (it IS authoritative source
        data, §11.4.95)

bridge:
  version: "1.0.0"

  # Workable items (§11.4.93 / §11.4.95)
  tracker:
    db_path: "docs/workable_items.db"
    id_prefix: "ATM"
    sync:
      command: "bash scripts/sync_issues_docs.sh"
      on_stage_complete: true
    external:
      - service: "clickup"
        list_id: "901818394542"
        credential_env: "CLICKUP_API_TOKEN"

  # LLM configuration (§11.4.196 / §11.4.198)
  llm:
    default_model: "fable"
    default_effort: "xhigh"
    providers:
      - name: "local"
        type: "native"
        backend: "llama.cpp"
        priority: 1
```

```yaml
      - name: "anthropic"
        type: "provider"
        priority: 10
      - name: "deepseek"
        type: "provider"
        priority: 20

  # Hardware constraints (§12.6 / §12.12)
  hardware:
    cores: 64
    ram_gb: 256
    vram_gb: 32
    safe_parallelism: 6        # Derived from cores/ram, never
        guessed (§11.4.6)

  # Design tokens (§11.4.216)
  design:
    tokens_path: "design/opendesign/tokens.css"
    platform_codegen:
      android: "app/src/main/res/values/tokens_colors.xml"
      web: "src/styles/tokens.css"
```

---

§7. Docs Chain Integration

The SpecKit-Superpowers Bridge's output artifacts — `spec.md`, `plan.md`, `tasks.md` — are §11.4.65-scope documents. They MUST be synchronized through the docs_chain engine (§11.4.106):

Documents produced by the bridge

Document	Stage	Export formats
`spec.md`	SPECIFY	`.md` + `.html` + `.pdf` + `.docx`
`plan.md`	PLAN	`.md` + `.html` + `.pdf` + `.docx`
`tasks.md`	TASKS	`.md` + `.html` + `.pdf` + `.docx`
`report.md`	IMPLEMENT (completion)	`.md` + `.html` + `.pdf` + `.docx`
`review.md`	REVIEW	`.md` + `.html` + `.pdf` + `.docx`

docs_chain context registration

The consuming project registers a docs_chain context for bridge artifacts:

```yaml
# .docs_chain/contexts/speckit.yaml
name: speckit_outputs
```

```yaml
description: "SpecKit bridge output synchronization"
sources:
  - path: "docs/speckit/current/spec.md"
    role: source
  - path: "docs/speckit/current/plan.md"
    role: source
  - path: "docs/speckit/current/tasks.md"
    role: source
exports:
  - format: html
    pandoc_args: ["--css", "constitution/styles/default-md.css"]
  - format: pdf
    weasyprint_args: ["--stylesheet", "constitution/styles/
        default-pdf.css"]
  - format: docx
fingerprint:
  method: sha256
  sidecar: "docs/speckit/current/.fingerprint"
```

Freshness enforcement

The §11.4.86 drift-proof fingerprint ensures that when any bridge artifact changes, the exports are regenerated before the next commit. The pre-build gate `CM-DOCS-EXPORT-SYNC` (§11.4.60) catches any stale export regardless of how the change was made. The §11.4.75 pre-commit git hook refuses a commit that touches `spec.md`/`plan.md`/`tasks.md` without also touching their `.html`/`.pdf`/`.docx` siblings.

---

§8. Auto-Propagation Hook

How `post_update_hook.sh` registers bridge capabilities

The §11.4.164 `post_update_hook.sh` is invoked by every consumer after every constitution pull. It detects what changed and registers it. For the SpecKit-Superpowers Bridge, the hook's internal classification maps these constitution paths to registration actions:

Changed path in constitution	Registration action
`skills/speckit-*`	Symlink into `<project>/.claude/skills/`
`plugins/helix/commands/speckit-*`	Regenerate per-agent slash commands
`scripts/speckit/*`	Validate `bash -n` + `sh -n` per §11.4.67

Changed path in constitution	Registration action
`actions/registry.yaml` (new actions)	Regenerate the `generated/` prefix-command manifests
`mcp/speckit-*.json`	`jq`-merge into `<project>/.mcp.json`

Detection flow

```
# Inside post_update_hook.sh (simplified):
detect_changes() {
  git -C "$CONST_DIR" diff --name-only ORIG_HEAD..HEAD 2>/dev/
        null || \
    git -C "$CONST_DIR" diff --name-only HEAD~1..HEAD
}

# When a changed path matches skills/speckit-*
install_skill() {
  local skill_dir="$1"
  local skill_name
  skill_name="$(basename "$skill_dir")"
  local target="$PROJECT_ROOT/.claude/skills/$skill_name"

  if [[ -d "$target" ]] && [[ ! -L "$target" ]]; then
    echo "WARNING: $target is a directory, not a symlink — left
        untouched (§11.4.122)"
    return 0
  fi

  ln -sf "$CONST_DIR/$skill_dir/SKILL.md" "$target"
  echo "  ✓ Registered skill: $skill_name"
}
```

The hook is idempotent

Running `post_update_hook.sh` multiple times is safe: symlinks are refreshed, directories are never deleted, commands are regenerated from the current registry. A missing `claude` CLI is reported honestly (the exact one-time commands to run are printed); every other agent's wiring proceeds independently.

§9. Constitutional Mandates Binding the Bridge

This table is the authoritative cross-reference: every constitution §-letter that binds the SpecKit-Superpowers Bridge, and what compliance means for the bridge implementation.

§-letter	Short name	What it means for the bridge
§1.1	Mutation-paired gates	Every bridge gate (`CM-SPECKIT-*`) ships a paired mutation that makes it FAIL
§11.4.5	Captured-evidence quality	Every bridge stage produces audio/video/sysfs/ffprobe/sink-side proof, never metadata-only
§11.4.6	No-guessing	Bridge generates fact-based decisions; `LIKELY`/`probably` forbidden in any bridge output
§11.4.8	Deep-web-research	PLAN stage cites ≥1 external research URL or "NO external solution found — original work"
§11.4.10	Credentials handling	Bridge NEVER hardcodes credentials; loads from `.env` files at runtime only
§11.4.15	Item-status tracking	Every bridge task carries a `**Status:**` line updated through the 6-stage lifecycle
§11.4.16	Item-type tracking	Every bridge task carries `**Type:** Bug | Feature | Task`
§11.4.17	Universal-vs-project	Bridge anchor lives at Layer 1 (universal); project config lives at Layer 2 (DATA)
§11.4.20	Subagent-driven	IMPLEMENT stage dispatches subagents; non-trivial tasks never run inline
§11.4.28	Decoupling	Bridge engine carries ZERO project literals; consumer config is DATA
§11.4.35	Canonical-root inheritance	Bridge rules travel through the 3-layer hierarchy
§11.4.43	TDD-fix-discipline	RED→GREEN→VERIFY→DOCUMENT cycle bound by every task
§11.4.44	Revision header	spec.md, plan.md, tasks.md carry `**Revision:** N` + `**Last modified:**`
§11.4.50	Deterministic consistency	Every bridge test runs N=3 iterations on identical artifact fingerprint
§11.4.54	ATM-NNN ticket ID	Every bridge-produced workable item carries a stable, auto-incremented ID
§11.4.58	Parallel PWU	

§-letter	Short name	What it means for the bridge
		Bridge tasks are parallel work units with file-scope manifests + isolated worktrees
§11.4.65	Universal MD export	Every bridge document exports to `.md` + `.html` + `.pdf` + `.docx`
§11.4.66	Interactive clarification	CLARIFY stage asks ≤5 targeted multi-choice questions, encodes answers back
§11.4.67	Shell parseability	Every bridge script (`.sh`) parses under `sh -n`, bash-only constructs wrapped in `eval`
§11.4.69	Sink-side evidence taxonomy	Every bridge PASS cites an evidence path matching the feature class from the closed taxonomy
§11.4.70	Subagent-default execution	IMPLEMENT stage dispatches per §11.4.70; inline only for ≤300-line single-file edits
§11.4.74	Catalogue-first	Bridge registers in `submodules-catalogue.md`; agents survey before scaffolding
§11.4.75	Mechanical enforcement	Bridge gate violation is caught by git hooks + pre-build sweep, not operator vigilance
§11.4.84	Working-tree quiescence	IMPLEMENT subagents run in isolated `git worktree`; no mutation-residue in commits
§11.4.85	Stress+chaos	Every fix ships stress + chaos test suites with captured evidence
§11.4.106	Docs chain	Bridge artifacts sync via docs_chain engine, not ad-hoc scripts
§11.4.107	Anti-bluff AV validation	Every media-output PASS proves liveness (n frames advancing), not a single stale screenshot
§11.4.108	Four-layer verification	Fix verified at SOURCE→ARTIFACT→RUNTIME-ON-CLEAN-TARGET→USER-VISIBLE layers
§11.4.115	RED-baseline + polarity	Every task carries a `RED_MODE=1→0` polarity switch with machine-written verdict pairs
§11.4.125	Code-review gate	REVIEW stage dispatches independent reviewers BEFORE pre-build sweep + main build
§11.4.134	Iterate-until-GO	REVIEW re-runs until ZERO findings + ZERO warnings
§11.4.142	Universal review	Every task diff crosses independent review — no "trivial change" exemption
§11.4.145	Impact research	

§-letter	Short name	What it means for the bridge
		PLAN stage includes 8-angle impact research: correctness, regression, latent, security, perf, safety, cross-feature, business-logic
§11.4.146	Reproduce-first-then-extend	Every task follows STEP-1 (RED+characterise) → STEP-2 (same-test GREEN) → STEP-3 (extend-all-cases)
§11.4.147	Crashed-agent respawn	Every IMPLEMENT subagent tracked in lifecycle registry; crashed→respawn until complete
§11.4.150	Deep multi-angle research	Every closure carries a documented ≥2-angle deep research pass, run in parallel
§11.4.151	Release-prefix naming	Bridge-managed release tags follow `<PREFIX>-<version>` per project convention
§11.4.157	GEMINI.md lockstep	Bridge anchor propagates to ALL 5 governance mirrors (Constitution+CLAUDE+AGENTS+QWEN+GEMINI)
§11.4.164	Auto-propagation hook	Bridge skills+commands installed via `post_update_hook.sh` on constitution pull
§11.4.186	Cross-document consistency	`speckit-analyze` enforces spec↔plan↔tasks consistency before any export
§11.4.194	Exhaustive code-review	REVIEW enumerates full input/scenario space + proves every assumption + cross-checks captured evidence
§11.4.197	Research completion	Every PLAN artifact maps to a tracked workable item driven to COMPLETED-and-wired or explicitly CLOSED
§11.4.201	Guard asserts real condition	Every bridge gate asserts the REAL condition with a self-validated golden-good/golden-bad oracle
§11.4.202	Reporting directives	ISSUE/BUG/TASK directives auto-create workable items; reports never evaporate into prose
§11.4.209	Fable-xhigh review	REVIEW agent dispatched on Fable model at xhigh effort (Opus xhigh fallback)
§11.4.224	TDD for ALL work	Every task — including new features — ships RED-first; coverage floor ≥85%
§11.4.227	Governance self-custody	Every `CM-SPECKIT-*` gate name is registered in the named-gate ledger under monotone ratchet
§12.6		

§-letter	Short name	What it means for the bridge
	Memory budget ≤60%	Bridge parallelism bounded by host RAM ceiling; no task launch exceeds budget
§12.12	Thread limit (RLIMIT_NPROC)	Bridge checks `ulimit -u` headroom before spawning parallel IMPLEMENT subagents

§10. Governance File Amendments

This section enumerates EXACTLY what must be added to each governance file in the constitution submodule. Following the 5-carrier lockstep rule (§11.4.157), every addition propagates to all five mirrors in the same commit.

constitution/Constitution.md

Add to the Table of Contents (insert in numerical order):

```
- [§11.4.NNB — SpecKit-Superpowers Bridge lifecycle](#114nnb-
        speckit-superpowers-bridge-lifecycle)
```

Add the full anchor text (from §2 of this document, §2 "Anchor text" subsection, verbatim), inserted in numerical order among the §11.4 family.

Add the recommended gate contract:

```
**CM-COVENANT-114-NNB-PROPAGATION** (literal `11.4.NNB`, block-
        start shaped per
§11.4.227(B), exactly ONCE per governance file across all 5
        mirrors per §11.4.157)
+ recommended gates `CM-SPECKIT-BRIDGE-SKILLS-PRESENT`
/ `CM-SPECKIT-BRIDGE-AUTO-DISCOVERY`
/ `CM-SPECKIT-BRIDGE-TDD-WIRED`
/ `CM-SPECKIT-BRIDGE-NANO-DECOUPLING`
/ `CM-SPECKIT-BRIDGE-CATALOGUE-REGISTERED`
/ `CM-SPECKIT-BRIDGE-CONSUMER-CONFIG`
+ paired §1.1 meta-test mutations (gate-code = separate work
        item).
```

constitution/CLAUDE.md

Add to the "Critical base rules restated" section, in the existing alphabetical anchor order:

```
### SpecKit-Superpowers Bridge lifecycle (§11.4.NNB)

The SpecKit-Superpowers Bridge is a core constitution technology
        stack —
specify→plan→tasks→implement→review lifecycle with decoupled nano-
        task
subagent execution and full TDD coverage. Every governed project
        inherits
it automatically; no per-project installation beyond
`post_update_hook.sh` is required. The bridge mechanizes what the
constitution mandates as prose: every task ships RED-first, every
        closure
lands four-layer test coverage, and every stage produces captured
        physical
evidence per §11.4.5/§11.4.69/§11.4.107.
```

constitution/AGENTS.md

Add the identical text from CLAUDE.md above, in the same position. The five-carrier lockstep rule (§11.4.157) requires byte-identical content across all mirrors. The propagation gate `CM-COVENANT-114-NNB-PROPAGATION` validates this at the block-start level.

constitution/QWEN.md

Add the identical text. Same constraints as CLAUDE.md and AGENTS.md.

constitution/GEMINI.md

Add the identical text. Same constraints as all other mirrors.

Block-start verification

After adding the amendment, verify that the propagation gate's block-start pattern matches exactly ONCE per file:

```
for f in constitution/
        {Constitution,CLAUDE,AGENTS,QWEN,GEMINI}.md; do
  count=$(grep -c '^### .*SpecKit-Superpowers.*11\.4\.NNB' "$f"
        || true)
  if [[ "$count" -ne 1 ]]; then
    echo "BLOCK-INTEGRITY FAIL: $f has $count block(s), expected
        exactly 1"
  fi
done
```

This check will become the `CM-ANCHOR-BLOCK-INTEGRITY` gate per §11.4.227(B).

---

§11. Catalogue Registration

Entry to add to constitution/submodules-catalogue.md

Under the **AI / Agent Tooling** section (or a new **Development Lifecycle** section), add:

```
### Development Lifecycle

- **[`speckit-bridge`](https://github.com/HelixDevelopment/
      speckit-bridge)** —
  SpecKit→Superpowers lifecycle bridge:
      specify→plan→tasks→implement→review
  pipeline with constitutional TDD, nano-task decoupling,
      subagent-driven
  execution, and full physical-evidence anti-bluff coverage at
      every stage.
  Inherited from the constitution submodule (§11.4.NNB). Consumed
      by reference
  (never copied) per §11.4.28.
```

If a dedicated organization repo is created (e.g., `HelixDevelopment/speckit-bridge` or `vasic-digital/speckit-bridge`), update the URL. If the bridge lives entirely in the constitution submodule (under `scripts/speckit/` and `skills/speckit-*/`), the catalogue entry points at the constitution repo itself with a path qualifier:

```
- **[`speckit-bridge`](https://github.com/HelixDevelopment/
      helix_constitution/tree/main/scripts/speckit)** —
  ...
```

helix-deps.yaml

The bridge ships a `helix-deps.yaml` in the constitution root (if it carries own-org dependencies) per §11.4.31:

```yaml
name: speckit-bridge
version: "1.0.0"
dependencies:
  - name: "workable-items"
    ssh_url: "git@github.com:HelixDevelopment/
        helix_constitution.git"
    ref: "v1.0.0"
    why: "SQLite workable-items DB for tracking bridge-produced
        tasks"
```

```
    layout: flat
  - name: "docs_chain"
    ssh_url: "git@github.com:vasic-digital/docs_chain.git"
    ref: "main"
    why: "Document export synchronization for spec/plan/tasks
        artifacts"
    layout: flat
```

---

§12. Validation & Verification

Pre-build gate integration

The bridge's pre-build gate (`CM-SPECKIT-BRIDGE-SKILLS-PRESENT`) is wired into the consuming project's `pre_build_verification.sh` under a new section (e.g., `DN-SPECKIT` or `DO-SPECKIT` depending on the next available letter pair). The gate asserts:

1. All 8 bridge skill directories exist under `constitution/skills/speckit-*`
2. Each contains a `SKILL.md` with the `---` frontmatter block and required `name:` + `description:` field
3. Each `SKILL.md` parses clean (all multi-line strings balanced, no truncated YAML)
4. The bridge orchestrator script exists at `constitution/scripts/speckit/bridge_lifecycle.sh` and is executable
5. The script passes `bash -n` AND `sh -n` parse checks (§11.4.67)

Paired meta-test mutation

The mutation test temporarily:

1. Renames `SKILL.md` → `SKILL.md.bak` in one skill directory → gate MUST FAIL
2. Strips the `name:` line from a frontmatter block → gate MUST FAIL
3. Injects a bash-only `>(...)` construct without `eval` wrapping → parse gate MUST FAIL
4. Restores → gate MUST PASS

The mutation test is registered in `meta_test_false_positive_proof.sh` under the bridge's section.

End-to-end consumer validation

A HelixQA Challenge (`CHALLENGE-SPECKIT-001`) validates the full consumer path:

1. **Setup** — a consumer project with the constitution submodule at a pinned commit containing the bridge anchor
2. **Onboard** — run `post_update_hook.sh` and verify all 8 skills are symlinked
3. **Specify** — issue `/speckit.specify` with a feature description, verify `spec.md` is produced with revision header + ≥40-char descriptions
4. **Plan** — run `/speckit.plan`, verify `plan.md` cites research references
5. **Tasks** — run `/speckit.tasks`, verify `tasks.md` contains nano-tasks with dependency edges
6. **Implement** — run `/speckit.implement`, verify subagents complete with captured evidence
7. **Anti-bluff** — verify every PASS verdict carries an evidence path (§11.4.69 `ab_pass_with_evidence`), verify no metadata-only PASS exists

The Challenge is registered in `tools/helixqa/banks/atmosphere.yaml` (or its project-equivalent).

Self-validation (golden-good / golden-bad)

Per §11.4.107(10), every bridge analyzer ships:

- **Golden-good fixture** — a valid `spec.md+plan.md+tasks.md` triplet that MUST PASS the consistency analysis
- **Golden-bad fixture** — a triplet with a spec-task that has no corresponding plan section; analysis MUST FAIL and pinpoint the orphan
- **Negative-control fixture** — two distinct features with similar-sounding subjects that MUST NOT be merged by the dedup analyzer

An analyzer that passes its golden-bad fixture is a bluff and the gate is itself a defect (§11.4.201).

---

§13. Implementation Phases

The implementation follows the §11.4.167 feature work-stream lifecycle on a dedicated branch (`feature/speckit-bridge`) with its own CoW clone and isolated build/test queue.

Phase 1 — Anchor landing (this document)

- [x] Research materials gathered (`request_with_materials.md` + design docs)
- [ ] This integration document finalized and committed
- [ ] Anchor text proposed, number allocated by constitution maintainer
- [ ] Propagation gate written, block-start verified across 5 mirrors

Phase 2 — Bridge engine scaffold

- [ ] `constitution/scripts/speckit/` directory created with `bridge_lifecycle.sh`
- [ ] `constitution/scripts/speckit/nano_task_decomposer.sh` — task DAG generator
- [ ] `constitution/scripts/speckit/tdd_harness.sh` — RED-first verdict-pair toolkit
- [ ] `constitution/scripts/speckit/spec_validator.sh` — cross-artifact consistency
- [ ] `constitution/scripts/speckit/schemas/bridge_config_schema.yaml` — consumer config JSON Schema
- [ ] All scripts pass `bash -n` AND `sh -n` (§11.4.67)
- [ ] All scripts documented per §11.4.18 (in-source block + `docs/scripts/` guide)

Phase 3 — Skill registrations

- [ ] 8 bridge skill directories created under `constitution/skills/`
- [ ] Each `SKILL.md` complete with frontmatter, workflow, compliance cross-references
- [ ] Plugin commands registered under `constitution/plugins/helix/commands/`
- [ ] `post_update_hook.sh` classification rules extended for `skills/speckit-*`
- [ ] `install_cli_agent_plugins.sh` extended for bridge slash commands
- [ ] All skills validated on a clean consumer project

Phase 4 — Catalogue + deps

- [ ] Bridge entry added to `constitution/submodules-catalogue.md`
- [ ] `helix-deps.yaml` declared at constitution root
- [ ] §11.4.31 recursive dependency graph validated
- [ ] `incorporate-submodule` helper verified against the manifest

Phase 5 — Gates + mutations

- [ ] 6 recommended mechanism gates written (`CM-SPECKIT-BRIDGE-*`)
- [ ] Each gate wired into the `pre_build_verification.sh`-equivalent sweep
- [ ] 6 paired §1.1 meta-test mutations written
- [ ] Propagation gate `CM-COVENANT-114-NNB-PROPAGATION` written and verified
- [ ] All gates registered in the §11.4.227 named-gate ledger
- [ ]

Golden-good + golden-bad + negative-control fixtures for every analyzer

Phase 6 — Consumer validation

- [ ] HelixQA Challenge `CHALLENGE-SPECKIT-001` written
- [ ] Challenge register in test bank
- [ ] Full consumer onboarding validated on a fresh project
- [ ] 6-stage pipeline run end-to-end on a real feature
- [ ] All evidence captured under `docs/qa/<run-id>/`

Phase 7 — Release

- [ ] §11.4.40 full-suite retest on both device targets (where applicable)
- [ ] §11.4.185 manual QA confirmation
- [ ] Constitution tag created + pushed to all upstreams (§11.4.113 no-force-push)
- [ ] Consumer project `.gitmodules` pointer bumped to new constitution HEAD